

Throughput-Optimized Networks at Scale

Conor James Green and Mithuna Thottethodi

Elmore Family School of Electrical and Computer Engineering, Purdue University

{green456,mithuna}@purdue.edu

ABSTRACT

Data center network design plays a critical role in AI training by supporting scaling to thousands of accelerators. An open problem, designing a near-optimal throughput-oriented network—topology, routing, and collectives—has not been achieved at scale and with broad applicability to physical or implementation constraints. We address this problem with a compelling use-case, Google’s TPU v4/5p supercomputer where the topology may be reconfigured to achieve higher all-to-all throughput, supporting large, parallelized AI training. We show that the existing TPU networks leave terabytes per second of throughput on the table and we fill that gap.

This paper presents Throughput-Optimized Networks at Scale (TONS), an automated network synthesis framework that meets the high-throughput demands of modern computing. TONS formulates topology synthesis as a linear optimization problem that maximizes a throughput-centric proxy metric, using theory and heuristics to scale to thousands of nodes. We further introduce a deadlock-free routing scheme compatible with limited virtual channels and optical switch faults, enabling the synthesized topologies to realize their predicted throughput gains in simulation. Evaluating uniform random and all-to-all traffic, TONS networks have a geometric mean speedups of $2.1\times$ and $1.6\times$ over the best TPU v4/5p torus variants.

1 INTRODUCTION

The rapid progress of artificial intelligence (AI) and large language models (LLMs) has been driven by scaling model size, dataset size, and training compute [47]. State-of-the-art models now reach billions to trillions of parameters [8, 10, 53, 56, 62, 81], pushing training from single accelerators to distributed execution spanning thousands of devices. At these scales, end-to-end throughput is frequently limited by sustained communication rather than peak compute, and modern training stacks explicitly co-design parallelization and scheduling with the network to achieve acceptable utilization [4, 44, 59, 65, 89, 93].

Distributed training uses data, tensor, pipeline, and/or expert parallelism, introducing synchronized collectives into the critical path. The parallelism techniques require frequent gradient synchronization via all-reduce or reduce-scatter+all-gather [43, 65, 69] and bandwidth-heavy all-reduce/all-gather between layers [59, 73]. Although overlap and pipelining can reduce exposed communication [39, 58], large-scale studies

still report regimes where communication becomes a dominant limiter and scaling efficiency diminishes as bandwidth per device drops [26, 59, 93]. Moreover, MoE-style routing induces token dispatch/gather phases implemented as all-to-all, which can dominate step time even with aggressive overlap [41].

Collective libraries continue to improve [11, 70, 88] but topology remains a hard constraint: no schedule can overcome fundamental bottlenecks from insufficient cut capacity, limited path diversity, or load concentration. This motivates a broad line of work showing that irregular and expander-inspired designs can deliver substantially higher throughput than classical structured networks [5, 77, 85, 92]. Many factors affect the end throughput for networks but analytical analysis on the topology provides strong upper bounds on performance. Recent research has pointed to the maximum concurrent flow (MCF) [71] of a topology to indicate the throughput as it represents the maximum routable/usable throughput [5, 92] and a tighter bound than topological metrics such as cut bounds (bisection bandwidth [14] or sparsest cut [54]) or link occupancy [23]. Many topologies have been hand-crafted or algorithmically designed to improve this value either directly or through proxies (e.g., diameter). Motivated by collective-heavy AI/ML training traffic, we focus on direct interconnects because prior works show direct topologies can deliver high aggregate throughput and utilization (at a fixed port budget) and remain robust across traffic patterns [77, 85, 92].

To exemplify scalability and implementability under strict constraints, we target Google’s TPU v4 and v5p AI training supercomputers [2, 45]. These hyperscale TPU clusters serve and train products for conventional services like YouTube, Gmail, and Google Maps as well as Google’s flagship AI model, Gemini [84]. TPU pods connect electrically wired 64-chip “cubes” using an optically reconfigurable interconnect (Palomar OCS and related designs) to build pod-scale direct networks [45, 50, 51, 63, 75, 83]. In practice, the control plane instantiates prismatic 3D tori (regular or twisted) [9, 45] and uses reconfiguration primarily for partitioning and switching between these variants despite substantial headroom for richer topology choices under the same physical constraints. Zu et al. describe additional operational constraints at this scale: routing is implemented via static forwarding

tables with a small virtual-channel budget for deadlock freedom, and fault tolerance requires offline routing under failures [94]. Evaluating an established network, this paper does not raise any ethical issues.

Despite clear opportunity, improving pod-scale fabrics faces three obstacles. (i) Solution space: even under strict degree and port constraints, the number of possible topology permutations grows super-exponentially while all-to-all style demand induces $\Theta(n^2)$ communicating pairs. (ii) Objective fidelity: directly optimizing end performance would require modeling routing realizability, deadlock avoidance, and collective scheduling inside the synthesis loop but optimizing weak proxies results in poor performance. (iii) Implementability: high-throughput designs are often irregular and inapplicable to many physical constraints [48, 77, 85, 90, 92], yet TPU routing must remain compatible with static forwarding, limited VCs, and OCS connectivity rules [94].

We propose Throughput-Optimized Networks at Scale (TONS), a network design framework that automatically generates topologies with optimal/near-optimal analytical throughput and implements deadlock-free routing to realize these gains, while maintaining OCS feasibility and routing constraints. TONS formulates topology construction as a mixed integer linear program (MILP) targeting a throughput-aligned proxy objective based on Leighton–Rao-style cut/flow foundations [23, 49], enabling scalable synthesis without simulation in the loop.

This work makes the following contributions.

- Linear programming formulation that generates TPU-feasible topologies optimized for an MCF-aligned proxy objective.
- Theory- and symmetry-based reductions that enable pod-scale topology generation.
- Deadlock-free routing algorithm with a small VC budget that achieves optimal/near-optimal routed throughput, supports fault tolerance, and balances VC load.

2 BACKGROUND

2.1 Analytical Models on Performance

Large-scale AI training and inference increasingly runs on distributed accelerator fabrics where step time depends on sustained communication throughput as much as compute. Parallel training mixes data/model parallelism, making bandwidth-heavy collectives (e.g., all-reduce, all-gather) and, in many workloads, dense reshuffles such as all-to-all performance-critical [5, 92]. Accordingly, we focus on *throughput* (the saturation point under concurrent demand) rather than single-message latency [14, 23].

We separate (i) topology-limited throughput (graph bottlenecks) and (ii) routing-limited throughput (load concentration under static routing) [14, 22]. Throughput-Optimized Networks at Scale uses proxies for (i)–(ii) to make synthesis

tractable, then validates candidates with explicit routability constraints and simulation (Sections 5–7).

A standard throughput objective is *maximum concurrent flow* (MCF): the largest λ such that every commodity in a traffic matrix can simultaneously send λ units subject to link capacities [71]. Directly optimizing topology for MCF is expensive, so we rely on topology-only bounds and routing-aware proxies. Per-node injection limits throughput by total egress capacity [14, 66], while cut constraints limit throughput by cut capacity (specializing to bisection under uniform all-to-all) [14, 54]. For deterministic routing, the inverse of the maximum directed edge load (*max channel load*) captures how routing concentrates demand and upper-bounds uniform throughput [14, 82]. Finally, cut-based quantities such as the sparsest cut provide scalable approximations to multicommodity throughput (e.g., via Leighton–Rao-style guarantees), and are most reliable when paired with routed analysis and simulation [23, 49, 54].

2.2 TPU v4/5p Pod Structure and OCS

Google’s TPU v4/5p pod interconnect uses optical circuit switching (OCS) to scale beyond a single electrically-cabled building block [1, 45]. The system is assembled from 64-chip *cubes* arranged as a $4 \times 4 \times 4$ 3D mesh. On its six faces, each cube has optical ICI links that connect to hardwired OCSes to realize a job-level inter-cube topology [45]. The OCS is software reconfigurable to directly connect/pair edges via a MEMS-based mirror, effectively creating a directly connected topology [63, 75, 83]. In production, Google deploys various configurations of 3D prismatic tori (PT) and prismatic doubly twisted tori (PDTT) as baseline job topologies up to 8192 nodes [45].

This setting imposes constraints uncommon in general DCN synthesis: fixed intra-cube wiring, fixed optical port budgets per cube face, and OCS-imposed connection restrictions (circuits connect only within a switch group). Moreover, forwarding is deterministic and configured per job (i.e., static routing tables) [45], so routing choices directly shape max channel load and achievable throughput under all-to-all or similar demand (Section 5). Google utilizes minimal hop XYZ dimension ordered routing (DOR) and datelines for deadlock freedom, performing heuristic or ILP-based route optimizations for equivalent distance paths. Because the network uses wormhole-style flow control, routing must also be deadlock-free. Cyclic channel dependencies, represented in a channel dependency graph (CDG) can be eliminated by turn restrictions and/or escape virtual networks [13, 18, 27]. Finally, at pod scale, failures are expected. Google devises “wild first routing” (WFR) where XYZ DOR paths that are disallowed due to a fault take a few hops through neighbors following the “sandwich rule” [94].

The baseline pod topologies (e.g., tori) have vertex/edge symmetry that can reduce synthesis and routing complexity by representing equivalent commodities as a small canonical set [82]. We use translations and/or reflections on the 3D coordinate grid to define a minimal canonical set S , a map $C(u)$ that returns the transformation taking u to $u_c \in S$, and an induced transform $T_u(v)$ applied to destinations, as formalized in Equation 1.

$$(u, v) \in E \iff (u_c, T_u(v)) \in E \quad \text{where } u_c \in S, \quad (1)$$

$$T_u : V \rightarrow V, \quad T_u(v) = v'.$$

2.3 Linear Optimization

Throughput-Optimized Networks at Scale casts topology synthesis and routing selection as linear optimization problems. A continuous linear program (LP) optimizes a linear objective over linear constraints with continuous variables. Mixed-integer and integer linear programs (MILPs and ILPs) introduce discrete (binary/integer) variables to express combinatorial structure (e.g., selecting edges, enforcing if-then logic), at the cost of worst-case NP-hardness [35]. For MILP, many solvers calculate the dual concurrently providing an upper (for maximization) bound on the objective. A standard scalability tool is relaxation: dropping integrality constraints yields an LP that upper-bounds the MILP objective and often provides a useful guide for synthesis but at the cost of global optimality.

For our sparse class of problems, the interior point method was the fastest and it relies on an AA^T for a constraint matrix A . This means that both the number of variables and constraints affect scalability. Finally, solver performance is governed not only by the number of variables and constraints, but also by sparsity. Interior-point methods (e.g., Gurobi’s barrier solver) repeatedly factor sparse linear systems whose cost depends on the fill-in induced by the constraint matrix. In practice, model formulations that preserve sparsity can be orders of magnitude faster and less memory-intensive [33]. This motivates the symmetry reductions and formulation choices in Section 4 and Section 5.

3 RELATED WORK

TONS performs optimization-based network synthesis for optically reconfigurable fabrics. Rather than selecting from a small family of rule-based templates (e.g., Clos or tori), TONS uses LP/ILP/MILP formulations and relaxations to optimize throughput-oriented proxy objectives and produce irregular, non-regular topologies. To our knowledge, no prior work directly *generates* topologies using an MCF-based linear optimization formulation. Most either generate candidates heuristically and evaluate them with MCF iteratively [12, 30, 37, 76, 77] or optimize different objectives (e.g., power/latency,

tail performance, or reconfiguration cost) [20, 28, 55, 67, 74, 78]. TONS targets the physical and control-plane constraints of TPU v4/5p pods—fixed intra-cube wiring, constrained OCS port groupings, and static forwarding—so we review work on optical reconfigurability, direct/irregular topologies, optimization frameworks, and throughput-centric theory.

Optical circuit switching and reconfigurable data-center fabrics – Helios [24] and c-Through [87] introduced hybrid electrical/optical fabrics that use OCS to accelerate high-bandwidth traffic patterns. Google’s Jupiter fabrics operationalize topology engineering with OCS and software-defined control at datacenter scale [63, 75]. These systems target rack/cluster networking, whereas TONS targets direct topology accelerator pods with stricter wiring/routing constraints and high-throughput workloads, including synchronized collectives.

Direct and irregular datacenter topologies – A broad line of work explores direct or server-centric DCNs beyond Clos hierarchies, including recursively defined DCell [32] and modular BCube [31]. CamCube advocates container-scale 3D tori with end-host forwarding [3], while SWDC and SpaceShuffle add structured randomness to improve path diversity and throughput with scalable routing [72, 90]. Scafida proposes an asymmetric, scale-free-inspired generator to support heterogeneity [34]. While these designs motivate moving beyond regular tori, they do not address TPU-style constraints (fixed intra-cube wiring, OCS port groupings) nor the static single-path forwarding model, and thus are not directly portable.

Optimization-based network design – REWIRE uses an optimization framework for unstructured DCN design but scales only to modest sizes [12]. COUDER optimizes OCS datacenters under traffic uncertainty and evaluates hop-count/throughput improvements under quasi-static reconfiguration [80], while Perseus jointly optimizes topology and cabling complexity but focuses on physical wire length rather than collective throughput [57]. In contrast, TONS targets all-to-all throughput under stringent structural constraints and scales to thousands of nodes while remaining compatible with static forwarding.

High-throughput graph constructions and throughput-centric theory – Irregular and expander-inspired DCNs such as Jellyfish [77] and Xpander [85], and low-diameter HPC networks such as Slim Fly [6], show that high expansion and path diversity can outperform structured designs in throughput. However, these systems typically assume packet-switched routers with flexible (often multipath) routing, whereas TONS operates under static single-path forwarding with a small VC budget. VL2 [29] popularized Valiant Load Balancing [86] as an oblivious mechanism to support broad traffic matrices, including under failures [91],

but its assumptions differ from TPU-style deterministic forwarding.

Finally, throughput under concurrent demand is analyzed by maximum concurrent flow and cut-based bounds: Leighton-Rao formalize approximate max-flow/min-cut relationships for uniform multicommodity flow [49], and Jyothi et al. advocate throughput-centric evaluation and clarify when cut metrics are predictive [23]. TONS adopts a cut-based proxy aligned with these foundations to enable scalable synthesis.

4 TOPOLOGY DESIGN

4.1 Topology Generation Overview

TONS synthesizes the optical inter-cube connectivity for a given job configuration under TPU/OCS wiring rules. Even with constant radix and structured port groupings, the number of feasible optical matchings grows combinatorially, making simulation-in-the-loop search impractical. We therefore optimize a throughput-aligned proxy based on the approximate sparsest cut for uniform all-to-all demand.

We start from an established MCF LP formulation and introduce topology connection variables so the program *generates* a topology rather than only evaluates a fixed graph. We then apply: (i) a TPU-specific reduction that shrinks the triangle-inequality footprint without restricting configurability, (ii) symmetry reductions that collapse equivalent commodities/constraints to enable pod-scale runs, and (iii) an iterative LP relaxation that accelerates synthesis while preserving feasibility. The output is an optical adjacency matrix M that is directly implementable via OCS configuration and serves as the input to the routing and deadlock-avoidance pipeline in Section 5.

4.2 Basic Topology Generation

We synthesize topologies that maximize a throughput upper bound under all-to-all demand. We use maximum concurrent flow (MCF) as the guiding proxy and cast synthesis as linear optimization: (1) start from an LP that captures MCF, (2) modify it to include topology connections as variables, and (3) impose TPU/OCS feasibility constraints modularly.

For (1) we utilize the dualized MCF formulation as defined by Leighton and Rao (LR) [49]. The LR formulation is intended to approximate the sparsest cut of a graph by (exactly) finding the MCF and is provided as (LR).

$$\begin{aligned}
 (LR) \quad \min \lambda &= \sum_{(i,j) \in E} d_{i,j} = M \cdot d \\
 \text{s.t.} \quad & \\
 &\sum_{i \in V} \sum_{j \in V, j \geq i} d_{i,j} \geq 1 \\
 &d_{i,j} - d_{i,k} - d_{k,j} \leq 0 \\
 &\forall \text{ distinct } i, j, k \in V
 \end{aligned}$$

$$d_{i,j} \geq 0 \quad \forall i, j$$

(LR) defines a semi-metric d or in simple terms, “apportions the smallest amount of total distance so that the cumulative distances between the source/sink pairs is not too small” [49]. The (LR) formulation is a function of a symmetric channel input graph, $M = (V, E)$, with edge set E or equivalently the (flattened) edge/adjacency vector, $M_{i,j}$. The objective value is the exact MCF, λ .

Accomplishing (2) cannot be directly performed with (LR) because it would introduce multiplication between the adjacency and distance variables in the objective. Furthermore, the objective is minimization so a direct substitution would find the edges and distance values with the minimal MCF. Therefore, we utilize the theory of duality for linear programs to relate the distance and edge variables by only linear operations and cause a maximization objective sense.

4.2.1 Dual-Based Formulation. In the primal formulation (LR), the RHS of equations are labeled b . After dualization, the formulation has a maximization sense and variables only have constant coefficients. By the theory of duality, the objective value of the primal and dual are equal for optimal solutions and for sub-optimal solutions, this dualized objective will be strictly less than the maximally achievable MCF [68]. For clarity, we write the dualized formulation in matrix-vector form.

$$\begin{aligned}
 \text{maximize}_{\mathbf{y}} \quad &\lambda = \mathbf{b}^T \mathbf{y} \\
 \text{s.t.} \quad &\mathbf{A}^T \mathbf{y} \leq \mathbf{M} \\
 &\mathbf{y} \geq 0
 \end{aligned}$$

In the primal, the rows of A correspond to variables $d \in \mathbb{R}^{n^2}$ and the columns of A correspond to the constraints (first column for sum of distances and other n^3 for triangle inequality). In the dual, A^T has the same meaning but swapping rows and columns. It introduces a new variable $\mathbf{y}$ that represents the constraints (demand satisfaction and triangle inequality). We now consider the edge set as variables, labeling $m_{i,j}$ for each edge (i, j) to distinguish from the previous user-input M , without creating a quadratic program. This formulation is now over two variables $\mathbf{y}$ and $\mathbf{m}$ and seeks to maximize the MCF, λ , represented by the objective.

Intuitively, the formulation would connect every single node, i.e. $m_{i,j} = 1 \forall \text{ distinct } i, j \in V$. Any constraints on topology construction may be added by independently constraining $\mathbf{m}$ to accomplish goal (3). Specifically, we constrain the $\mathbf{m}$ to valid TPU v4/5p topologies that follow: reflexivity, symmetry, and valid optical connections. Reflexivity and symmetry are provided in Table 1 C1-2 but in practice, handled in code by directly substituting $m_{i,i}$ with 0 and any $m_{i,j}$ where $i > j$ with $m_{j,i}$. Valid optical connections follow OCS connectivity between (x, y, z) co-ordinates as described in Section 2.2 to define the total valid connections,

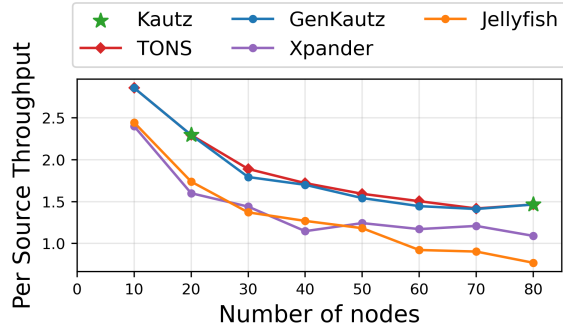


Figure 1: Analytical throughput of directed, regular four radix topologies from literature (Kautz [48, 79], GenKautz [40], Xpander [85], and random/Jellyfish [77]) versus topologies generated by our synthesis formulation, TONS. For each size, the y-axis is the maximum concurrent flow multiplied by the number of nodes (scale invariant metric).

$L_{valid} = L_{optical,X} \cup L_{optical,Y} \cup L_{optical,Z}$ (C3 in Table 1). Let matrix B and vector p represent these valid topology constraints. By the RHS of the primal, $b = [1, 0, \dots, 0]$ so the objective can be simplified to $b^T y = y_0 = \lambda$. For later ease of understanding, we label all y associated with the primal triangle inequality for i, j, k as $y_{\Delta i,j,k}$.

With these modifications, this formulation accomplishes: (1) the objective is exactly the MCF, (2) it is a linear program with edge connection variables, m , and (3) constraints on m can be added/removed without affecting (1) or (2). We label this complete, MILP the name of the paper, Throughput-Optimized Networks at Scale (TONS), and is provided in matrix-vector form (TONS) and, with subsequent improvements, in Table 1.

$$\begin{aligned}
 (TONS) \quad & \max \lambda \\
 & \text{s.t.} \\
 & A^T y - m \leq 0 \\
 & Bm \leq p \\
 & y \geq 0, m \in \{0, 1\}
 \end{aligned}$$

4.2.2 Empirical Validation of Approach. In a preliminary analysis to validate the (TONS) formulation, we compare against known good topologies with constraints (four radix, undirected) matching prior works [92]. The matrix B and vector p were set to keep the maximum out and in degrees of m less than four and the symmetry constraint was excluded. We evaluated the MCF of four commonly cited topologies in the data center network (DCN) or AI/ML domain: Kautz [48, 79], GenKautz [40], Xpander [85], and Jellyfish (random) [77]. Kautz graphs deterministically generated for $N = (1 + r)r^m$ for N nodes, r radix, and m given parameter. GenKautz graphs are a generalization of Kautz graphs to apply to any combination of N and r . Xpander graphs

start with a small, r regular graph and perform “lifts” to the desired size. Jellyfish topologies are degree-bounded random graphs.

We generated directed, four radix direct topologies for 10 to 80 nodes—includes two instances of Kautz—and plot as a size invariant metric, per source injection rate, in Figure 1. For random, for each size, we created 100 topologies and took the highest value. We compare the aforementioned approaches to TONS-generated topologies, generated through linear optimization to directly improve the MCF and include them in Figure 1. A trend is clear: for all sizes in this sweep, TONS topologies are equal to or better by a few percent than all other approaches. For node sizes without a Kautz topology, TONS generated novel and strictly superior topologies, and for this example, generated all topologies in less than a day. These results validate the formulation and give “proof of opportunity” to our approach. We now target the obstacles with LP-based topology generation: *scaling* up to 100× the size of these preliminary results and *implementing* a full network stack for routing, deadlock avoidance, fault-tolerance, and collective communication.

4.3 Formulation Scaling

4.3.1 One-Leg Reduction. The first obstacle to scalability in (LR) is the $\Theta(n^3)$ family of triangle inequalities. Let L_{valid} denote the set of possibly connected links; under TPU v4/5p constraints, $|L_{valid}| = \frac{n}{64}$. We reduce the variable $y_{\Delta i,j,k}$ footprint to $O(n^2|L_{valid}|)$ by only instantiating $y_{\Delta i,j,k}$ s.t. $(i, k) \in L_{valid}$, which preserves correctness while substantially reducing memory in practice. Though $|L_{valid}| = O(n)$ for cube face nodes, if connections become precluded iteratively (Section 4.3.3) then it becomes $O(1)$.

We explain this concept in the primal (LR) because it is easier to comprehend. The idea is that for an optimal solution to (LR), at least one triangle inequality must be tight because one intermediate k upper bounds the distance $d_{i,j}$ for all pairs. If this is the case then which k performs this can be limited to a subset $K = \{k | m_{i,k} = 1\}$. This line of thinking is similar to Nguyen and Minoux [61] but has stronger guarantees applied to optimization. Due to space, we present the full proof of this idea for the primal in Appendix A and utilize it in the dual. We call this variable reduction, “one-leg” for the one active leg of the triangle inequality. We apply this idea to the dualized formulation (TONS) by setting all $y_{\Delta i,j,k} = 0$ when $(i, k) \notin L_{valid}$ as seen in C5 of Table 1. In practice, we directly set $y_{\Delta i,j,k} = 0$ for known invalid connections (i, k) .

4.3.2 Vertex Symmetry. We further scale synthesis by enforcing edge symmetry, analogous to the symmetry reductions used for TPU routing Section 2.2. This collapses many equivalent edge variables into a small canonical set, reducing the number of decision variables without changing the induced topology class.

Table 1: Constraints + Objectives for TONS Topology Generation

Label	Objective (O)/Constraint (C)/Variable (V)	Description
O1	maximize λ $m_{i,y}$	MCF maximization
C1	$\forall i \in R, m_{i,i} = 0, y_{\Delta i,i,*}, y_{\Delta i,*,i}, y_{\Delta *,i,i} = 0$	Ignore self-adjacency
C2	$\forall i, j \in R, m_{i,j} = m_{j,i}$	Link symmetry
C3	$\forall i, \sum_{x \in L_{optical,X}} m_{i,x} = 1, \sum_{o \in L_{optical,Y}} m_{i,y} = 1, \sum_{z \in L_{optical,Z}} m_{i,z} = 1$	X, Y, Z link limitations
C4	$\forall \text{ distinct } a, b \in R \quad \lambda - \sum_{k \in R} y_{\Delta a,b,k} + \sum_{j \in R} y_{\Delta a,j,b} + \sum_{i \in R} y_{\Delta i,a,b} - m_{a,b} \leq 0$	Dualized LR
C5	$y_{\Delta i,j,k} = 0 \quad \forall (i, k) \notin L_{valid}$	One-Leg
C6	$\forall j, m_{i,j} = m_{i_c, T_i(j)} \quad \forall i \text{ s.t. } T_i(i) = i_c$	Edge Symmetry
C7	$\forall j, k, y_{\Delta i,j,k} = y_{\Delta i_c, T_i(j), T_i(k)} \quad \forall i \text{ s.t. } T_i(i) = i_c$	Edge Symmetry
C8	$\lambda \geq f+1/32 R $	Fault-tolerance
V1	$m_{i,j} \in [0, 1] \text{ or } m_{i,j} \in \{0, 1\}$	Adjacency matrix
V2	$y_{i,j,k} \in [0, 1]$	LR dual variables

We define a canonical set, S , to be one cube (64 nodes) which yields canonical variables $m_{i_c,j} \quad \forall i_c \in S, j \in V$. Then by translational symmetry, all non-canonical equivalents, $m_{i,j}$, use translational symmetry to map their canonical representation, $m_{i,j} = m_{i_c, T_i(j)}$. This equality is given in C6 in Table 1 but in practice it is achieved by only creating canonical variables and reconstructing all others on demand.

Applying symmetry not only reduces the number of variables from $O(n^2|L_{valid}|)$ to $O(n|L_{valid}|)$ but also reduces the number of constraints. This symmetry makes all constraints (C1-C5 in Table 1) for a non-canonical source i redundant and as such, reduces the number of constraints from $\Theta(n^2)$ to $\Theta(n)$. This reduces the overall complexity (Section 2.3) from $O(n^4|L_{valid}|)$ to $O(n^2|L_{valid}|)$, an n^2 reduction. Applying this technique allows the formulation to scale to the largest topology configurations and as will be shown, does not significantly reduce the performance of the synthesized topologies.

4.3.3 Integrality Relaxation. The previous two scaling techniques profoundly reduce the complexity of the formulation but there are no polynomial algorithms to solve MILPs in general and the time to find solutions explodes rapidly. To overcome this final obstacle, we apply a heuristic algorithm to iteratively solve a relaxed LP of (TONS). While there is little theoretical grounding to this heuristic, if the binary constraint on the adjacency matrix is relaxed to continuous (same $[0, 1]$ bounds) then the problem becomes a linear program and much easier to solve.

Essentially, many MILP solvers do this under the hood by first solving the relaxed LP and using an intelligent branch-and-bound algorithm to re-gain integrality for all binary/integer variables [7, 33] but it is computationally difficult. Therefore, we solve the linear program and greedily set adjacency matrix values. We select *interval* number edges per LP solve by choosing the (i, j) edges with the highest $m_{i,j}$ values. The choice of *interval* affects solution quality so we often set that

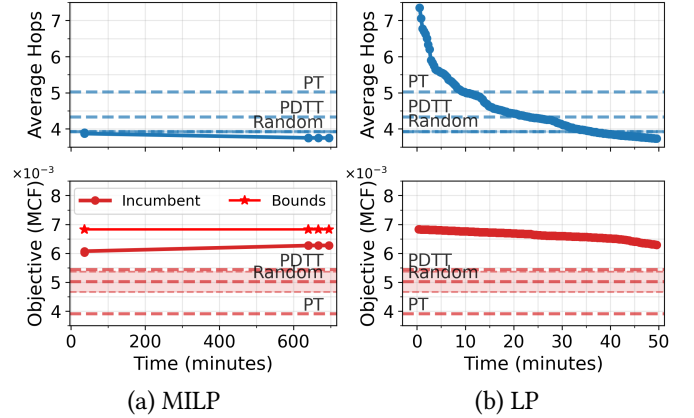


Figure 2: Progress of MILP (a) and LP (b) (non-symmetric) variants of TONS average hops (blue) and MCF objective (red) over time for a 256 node configuration. For MILP, each dot is an incumbent solution and each star is the (dual) bound. For LP, each dot is the objective value for each solution (final dot has binary edges). Included is TPU-applicable random with standard deviation shaded.

value to one but it is included for completeness. Intuitively, this is forcing integrality on the most “critical” edges for that solution instance. Due to space, the complete algorithm for this relaxed approach is provided in Appendix B.

For small topologies, the LP results in topologies with lower MCF by a few percent. However, the greedy, iterative LP approach has significant speed and memory footprint improvements over the MILP variant. Due to the difficulty of the MILP formulation, it cannot find quality solutions within reasonable time and compute/memory constraints. As such, in practice, the LP results in superior topologies.

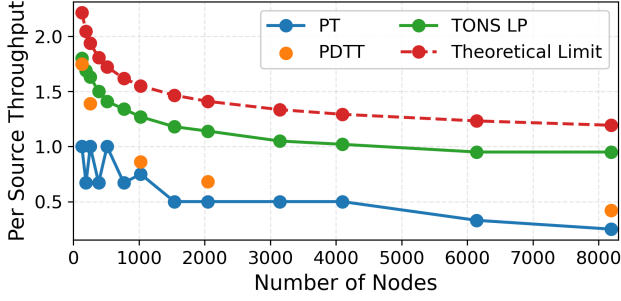


Figure 3: Per-source injection rate (MCF time number of nodes) for best PT, PDTT, and TONS sweeping topology sizes. Theoretical limit (dashed) is based on size and radix but unrealizable for TPU/OCS constraints.

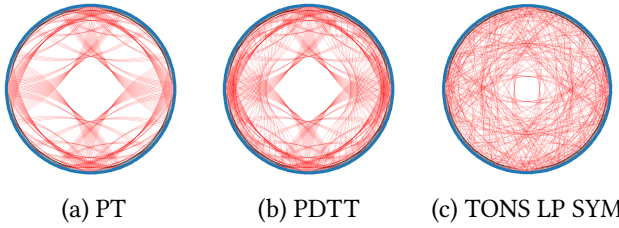


Figure 4: Visualization of the 256 node topologies between nodes (blue) with electrical (black) and optical (red) links. Clockwise around the circle, the nodes are grouped by cubes and ordered by z, y, x (ascending).

4.4 Resultant Topologies

We generate topologies for all sizes¹ for four variants of TONS: MILP, MILP with symmetry, LP, and LP with symmetry. We use AMD EPYC 7763 “Milan” CPUs with up to 256GB of memory and Gurobi version 12.0.0. An example of the average hops and MCF objective changing over time for a 256 non-symmetric synthesis run is plotted in Figure 2. We see that both the MILP and LP comprehensively beat the heuristic baselines (PT, PDTT) and the standard deviation for a random (TPU constrained) topology. Furthermore, we see the benefit of the iterative LP approach: faster topology synthesis with little objective loss. While these non-symmetric solutions take hours, the symmetric LP variant completes in 3 minutes for 256 nodes. At 8192 nodes, it takes five days.

Only LP with symmetry scales to the complete 8192 nodes and we plot the per source injection rate (number of nodes times MCF) versus PT and PDTT in Figure 3. In Figure 3, we also include a theoretical bound on the per source injection rate for any six radix, undirected graph as derived by Basu et al.: $\lambda \leq r / \log_r(n)$ for n nodes and radix r . This theoretical bound does not obey TPU v4/5p constraints but is true upper bound to provide context. TONS topologies not only

¹While PT and PDTT topologies differ based on the job dimensions due to their algorithmic construction, TONS topologies are the same regardless of the configuration.

have higher analytical throughput than the baselines but scale well, matching the curvature of a non-realizable upper bound. A complete table of analytical metrics is provided in Appendix C.

To give some idea about what causes these higher MCF values, we visualize one size (256 nodes) in Figure 4. The prismatic torus baselines concentrate optical connectivity into regular patterns, whereas TONS-generated designs select optical edges to improve multiple cuts simultaneously rather than optimizing a single bisection. This is precisely what the MCF proxy encourages: instead of maximizing one/two bottlenecks as in twisted tori, the optimizer improves the worst bottlenecks, both cut and link utilization bounds. Visually, the topologies in Figure 4 suggest that edge selection differs significantly across designs even when degree is identical. For reference, the bisection improved by PDTT over PT by design [9] is across the tightest two bisection cuts, 45° slanted diameters in Figure 4(b).

4.5 Alternative Formulations

Alternative MCF Formulations. Alternative MCF formulations assume a fixed path set derived from an input graph [71]. Here we synthesize the graph itself, so a path-based formulation would require either (i) enumerating paths in a graph that does not yet exist or (ii) iteratively regenerating paths as the topology changes, tightly coupling synthesis and routing at prohibitive cost.

A node-edge formulation is also problematic: constraint existence depends on whether an edge exists and encoding this cleanly introduces products between edge-selection and flow variables, yielding a non-scalable quadratic program [17]. Likewise, formulations that explicitly model all $n(n-1)$ commodities require variables for each commodity over each edge and flow conservation at every node, leading to $O(n^3)$ -scale flow variables/constraints in the constant-radix regime, far larger than our reduced formulation. Finally, the node-arc MCF variant that defines only n commodities (max flow from each source) is appealing [71], but we did not find a symmetry-compatible instantiation that preserves the reductions required for pod-scale synthesis.

Cut-Based Approaches. An alternative throughput-oriented LP is the sparsest cut [54]. At first glance, this seems well-suited to Benders decomposition [64] or cutting-plane methods [46] where critical cuts are added lazily. In practice, however, adding cut constraints iteratively requires prohibitively many rounds: LR-style relaxations typically have $\Theta(n^2)$ tight triangle constraints (for each (i, j) , some k attains $d_{i,j} = d_{i,k} + d_{k,j}$), suggesting on the order of n^2 constraints may be needed before convergence. Enumerating all cuts is intractable and prior cut-enumeration approaches scale only to very small networks (e.g., ≤ 30 nodes) [28].

Algorithm 1 Allowed Turns (AT)

```
1: Input:  $G$  ▷ Topology
2: Initialize:  $\bar{D} \leftarrow \text{complete\_cdg}(G)$ 
3: Initialize:  $A \leftarrow \emptyset$  ▷ Allowed turns
4: if robust then
5:    $S_0, S_1 \leftarrow \text{ocs\_disjoint\_spanning\_tree}(G)$ 
6:    $A, \bar{D} \leftarrow \text{add\_turns}(A, S_0, \bar{D}, \text{force\_vc}=0)$ 
7:    $A, \bar{D} \leftarrow \text{add\_turns}(A, S_1, \bar{D}, \text{force\_vc}=1)$ 
8: end if
9:  $S \leftarrow \text{spanning\_tree}(G)$  ▷ Enforced routability
10:  $A, \bar{D} \leftarrow \text{add\_turns}(A, S, \bar{D}, \text{force\_vc}=0)$ 
11:  $T' \leftarrow \text{prioritized\_turns}(G)$ 
12:  $A, \bar{D} \leftarrow \text{add\_turns}(A, T', \bar{D}, \text{single\_turn}=\text{True})$ 
13:  $A, \bar{D} \leftarrow \text{add\_turns}(A, T', \bar{D})$ 
14:  $P = \text{BFS}(G, A)$  ▷ Set of all deadlock-free paths
```

5 DEADLOCK FREE AND FAULT TOLERANT ROUTING

The topologies synthesized in Section 4 improve analytical throughput proxies but are not directly implementable using torus-specific routing rules. To realize these gains in a TPU-style lossless fabric, we must produce (i) static forwarding tables selecting a single path per source–destination pair, (ii) a VC assignment within a small VC budget, and (iii) fault-tolerant reachability under single-OCS failures.

Making an *arbitrary* routing function deadlock-free within a bounded number of VCs is NP-complete, but it is often possible to construct a *good* deadlock-free routing using a limited VC budget [15]. Prior work (e.g., Nue) integrates VC allocation and CDG maintenance while committing to routes early, but the resulting routing functions can be overly conservative and throughput-suboptimal [15]. Throughput-Optimized Networks at Scale instead decouples deadlock freedom from route selection: we first construct a large deadlock-free candidate path set using an *allowed-turn* construction on the channel dependency graph (CDG), then solve a throughput-maximizing ILP to choose one path per flow. As shown in Section 7.4, this achieves near-optimal throughput relative to an unconstrained (deadlocky) routing baseline, while remaining implementable and extensible to fault tolerance.

5.1 Allowed Turns and All Paths

Arbitrary topologies complicate deadlock freedom because simple rule-based schemes (e.g., dimension ordering + datelineing used for tori) do not generalize: a fixed rule can strand some source–destination pairs without a valid path. A common approach allocates routes to VCs as layered virtual networks using CDG-based sufficient conditions (acyclicity), but it provides no guarantee on the number of VCs required [16, 52]. Nue demonstrates that one can target a fixed VC budget by routing while maintaining a CDG [15], but early route commitments reduce path diversity and can degrade throughput.

TONS first constructs a deadlock-free turn set and all deadlock-free paths then selects globally optimal routes from

Algorithm 2 add_turns

```
1: Input:  $A$  ▷ Allowed turns
2: Input:  $T$  ▷ Turn set
3: Input:  $\bar{D}$  ▷ Complete cdg
4: Input: single_turn, force_vc ▷ Optional
5: for  $t \in T$  do
6:    $\bar{T} \leftarrow \text{turns\_with\_vcs}(t, \text{force\_vc})$ 
7:   for  $\bar{t} \in \bar{T}$  do
8:     if  $\neg \text{deadlocky}(\bar{t}, \bar{D})$  then
9:        $A \leftarrow \bar{t} \cup A$ 
10:       $\bar{D} \leftarrow \bar{t} \cup \bar{D}$ 
11:      if single_turn then
12:        break ▷ break to next base turn
13:      end if
14:    end if
15:  end for
16: end for
17: return  $A, \bar{D}$ 
```

that set. Starting from the complete CDG $\bar{D}$ over VC-labeled channels $\bar{E}$, we greedily add turns into a global allowed set A only when the insertion preserves acyclicity; any routing restricted to A is deadlock-free by construction. Algorithm 1 summarizes the approach and add_turns (Algorithm 2) implements the guarded insertion rule. We denote directed edges by $e_{i,j} \in E$ and VC-labeled channels by $e_{i,j,v} \in \bar{E}$; base turns are $(e_{i,j}, e_{j,k}) \in T$ with VC-labeled counterparts $(e_{i,j,v_0}, e_{j,k,v_1}) \in \bar{T}$. Compared to Nue, AT is lightweight because it operates on turns (not flows) and defers route selection to the routing ILP in Section 5.3. With symmetry enabled, we add/reject turns in symmetry classes and accept a class only if no member introduces a CDG cycle.

Because turn addition is greedy, the insertion order matters. We evaluate three prioritization heuristics in Section 7.4: *APL* (“all path list”) orders turns by decreasing frequency across the set of all candidate paths; *CPL* (“chosen path list”) orders turns by decreasing frequency in the single-path routing produced by the ILP; and *Random* adds turns in arbitrary order. APL yields the highest path diversity but CPL yields the lowest maximum channel load amongst these variants and is used for TONS/AT results under 4096 nodes (faster Random used for larger).

To guarantee routability, we seed A with a spanning-tree turn set as in Nue [15]: we build a tree rooted at a central node and add its up/down turns to VC0 (Algorithm 1, lines 9–10), ensuring at least one path between any pair (though not necessarily minimal). To retain path diversity and reduce bias from the prioritization order, we initially allow at most one VC-labeled instance $\bar{t}$ per base turn t (Algorithm 1, line 12; Algorithm 2, lines 12–13) before expanding to all admissible VC assignments.

5.2 Fault Tolerance

TPU v4/5p pods must remain operational under optical faults [94].

We adopt a conservative model consistent with TPU practice: an OCS fault disables all links routed through that OCS,

the fault is detected and broadcast before job execution, and routing tables are selected accordingly [45].

We augment all-path discovery (Algorithm 1) so that the candidate path set contains deadlock-free backups under any single OCS failure. A sufficient condition is the existence of t OCS-disjoint spanning trees: then at least $t - 1$ OCS faults can be tolerated while preserving connectivity via a tree that avoids the failed OCS. We use Nash-Williams [60] provided in Equation 2 as a sufficient condition for t edge-disjoint spanning trees and extend the argument to OCS-disjointness under TPU’s structured OCS grouping.

$$\sum_{C \in P} E(C) \geq t(k - 1). \quad (2)$$

In our setting, we connect this condition to a lower bound on the number of distinct OCS links crossing any partition as a function of MCF, yielding a conservative requirement of the form $\lambda \geq \frac{t}{32n}$ for n nodes and t OCS-disjoint spanning trees. We derive the complete proof of this property in Appendix D but the proof sketch using worst-case analysis is as follows. The MCF, λ , is greater than or equal to the cut bound and implies the minimum number of edges leaving a cube or groups of cubes and similarly, the lower bound on number of OCS distinct edges. We then perform algebra on Equation 2 to derive a conservative lower bound on λ in terms of the number of nodes and possible faults.

During topology generation, we encode this as constraint C8 in Table 1 for a user-specified budget for number of tolerable faults, f , via $f = t + 1$. All synthesized TONS topologies satisfy this property and empirically many support substantially more than one OCS fault. We label this fault-tolerance scheme *robust*; it is implemented in the AT stage lines 4-8 of Algorithm 1. For the single-fault model ($f = 1 \implies t \geq 2$), we find two OCS-disjoint spanning trees using a concurrent BFS rooted at hop-distance antipodes: the trees are grown simultaneously while marking an OCS as consumed when used by one tree². With a *robust* allowed turns set, A , sets of all paths are generated for each possible fault and routed using the ILP (Section 5.3) similar to Google’s WFR [94].

5.3 Routing

Given the deadlock-free candidate path set produced by AT, we must select exactly one path per source-destination pair (static forwarding). We choose routes to minimize congestion using *maximum channel load* as the objective, since it directly upper-bounds achievable uniform throughput: if the most-loaded channel carries $L_{\max}$ routes, then under uniform demand the per-flow rate is at most $1/L_{\max}$. This matches the

routing objective used for PDTT in Zu et al. [94] and is standard in throughput-oriented routing formulations [28, 82].

Our ILP uses one indicator variable per candidate path; selecting a path increments the load variables of every edge on that path. A scalar variable $L_{\max}$ is constrained to be at least every edge load, and the solver minimizes $L_{\max}$. As in prior work, the quality of the solution depends on the provided candidate set: shortest-path sets are often sufficient on regular tori, while the AT-generated minimal all paths set is essential for irregular topologies to preserve deadlock-freedom under VC constraints. Empirically, for both torus baselines and TONS topologies, the routed bound is typically tight relative to analytical continuous upper bounds, indicating that routing is not the dominant limiter in most configurations (Section 7.4).

5.4 VC Allocation

After route selection, the chosen paths and allowed turns list are used to allocate VCs to turns in the path. We consider each path individually and perform a breadth-first search along the complete CDG to find the allowed VC transitions/selections. A naïve approach biases towards VC 0. The BFS starts at VC 0 and traverses along VC 0 until a turn is disallowed when it transitions to VC 1 and continues trying to allocate to VC 0. However, we found this caused significant load imbalance between the VCs so we propose an online load balancing algorithm. As turns are allocated to VCs, the count of hops per VC is maintained. Before beginning VC allocation for the next path, the VC with the lowest hop count is marked “priority” and the BFS checks that VC for all turns in that path. We find this achieves near perfect balance (Section 7.4).

6 EVALUATION METHODOLOGY

We evaluate TONS using complementary analytical metrics and cycle-level simulation. Our experiments target the paper’s three claims: (i) synthesized topologies improve throughput-oriented proxies under TPU/OCS constraints, (ii) allowed-turn routing with a small VC budget preserves most of the attainable throughput while remaining deadlock-free, and (iii) these gains translate to all-to-all-style communication without degrading all-gather/all-reduce. Because TPU pods are not externally reconfigurable for arbitrary topologies or routing in practice [1], we rely on analytical evaluation and simulation.

6.1 Simulation of Networks

We simulate selected configurations using Chiplet Network Simulator (CNSim) [25], a cycle-accurate, packet-parallel simulator validated against gem5 [21] and BookSim2.0 [42] for synthetic traffic. We extend CNSim to ingest arbitrary

²We note that constructing many disjoint trees efficiently is nontrivial. We suspect matroid intersection extensions of Edmonds’ algorithm are appropriate [19] but leave this for future work.

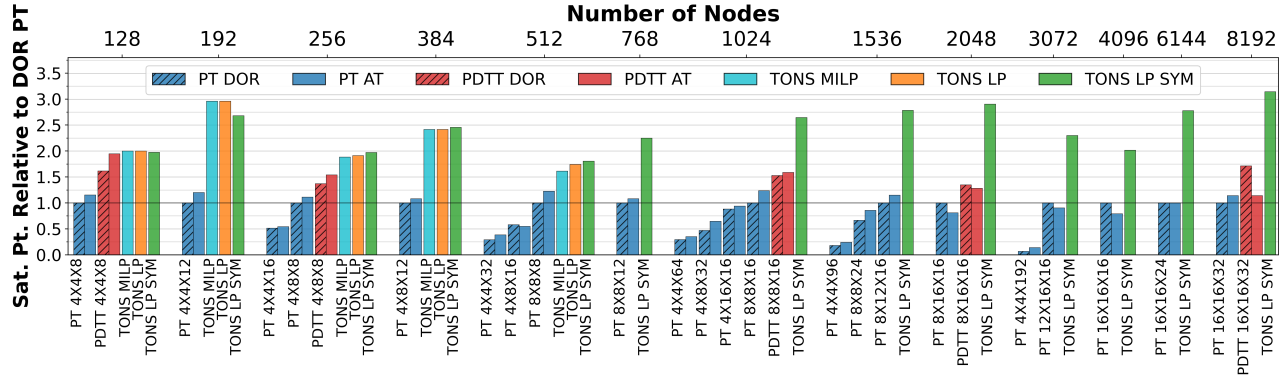


Figure 5: Relative saturation points (higher is better) for uniform random traffic simulations normalized to the best PT and DOR (blue hashed). DOR (hashed) and AT (solid) routing were applied to tori while TONS uses AT.

Table 2: Simulation Parameters

Parameter	Value
Clock frequency	1.05 GHz
Link bandwidth	128 GB/s (unidirectional)
Link latency	50 cycles electrical, 25 cycles optical
Router latency	25 cycles
Injection latency	25 cycles
Router radix	6
Flit width	128 B
Total & escape VCs	4 & 2
Buffer slots per VC	200 flits

adjacency matrices, static routing tables, and per-flow escape-VC assignments. We model 4 VCs total and reserve 2 as deadlock-free escape VCs [18].

Simulation Parameters. We tune parameters to match published TPU v4/5p values where available: TPU v5p clock and link bandwidth rounded to the integer flits per cycle [2]. For link latency, we conservatively estimate propagation from published system imagery [45] for a 5 m maximum cable length, <100 ns router delays, and 10-100 ns of delay for electrical links [83]. We roughly assume a 25 cycle injection latency. Buffers are sized to sustain the modeled bandwidth delay product with margin.

6.1.1 Saturation Point. We measure network throughput by simulating uniform random traffic in CNSim [25]. CNSim sweeps injection rates and reports saturation at the first observed timeout (flits sent > flits received). We use an injection-rate step of 0.01; other simulator parameters follow Table 2.

6.1.2 Collective Communication. We evaluate all-gather, all-reduce, and all-to-all using existing schedulers. For all-gather and all-reduce, we use MultiTree [38] (scales to our largest topologies and achieves near-optimal schedules). For all-to-all, we use the optimization-based formulations of Basu et al. [5]: decomposed-MCF at sizes where it scales, and pMCF

(with symmetry) at larger sizes. We report schedule quality as link utilization (epochs under cumulative link bandwidth). For a subset of configurations, we translate link-by-link transfer schedules into traces (MSCCLang-style XML [11] and netrace [36]) and simulate them in CNSim [25] to validate analytical expectations.

6.1.3 Fault Tolerance. Our fault model follows Google’s description for TPU v4/5p pods [45]: a fault disables all links routed through one OCS, at most one OCS faults at a time, and the fault is known before job execution. For each of the 48 single-OCS fault scenarios, we load the corresponding fault-avoiding routing tables and measure the saturation point under the same traffic and simulator settings.

7 RESULTS

We evaluate TONS via analytical metrics and cycle-level simulation. The primary result (Figure 5) is the saturation-throughput improvement of TONS topologies under AT routing. We then report (i) collective-communication utilization, (ii) robustness under OCS faults, and (iii) routing ablations that justify our AT turn prioritization and VC load balancing.

7.1 Saturation Point

We quantify throughput using uniform random traffic simulation, reported as the saturation point. Figure 5 compares baseline structured topologies (PT and PDDT) against TONS-generated designs across a wide range of node counts and job shapes. For each configuration we evaluate tori with both (i) baseline dimension-ordered routing (DOR) where applicable, and (ii) TONS’s allowed-turn routing (AT), isolating the contribution of routing from that of topology. All values are normalized to the best PT+DOR saturation point for that job configuration.

TONS topologies deliver large throughput gains across scales. Depending on configuration, TONS improves saturation by roughly $1.6\times$ – $3.1\times$ over the best structured torus baseline. Overall, TONS has a geometric mean improvement

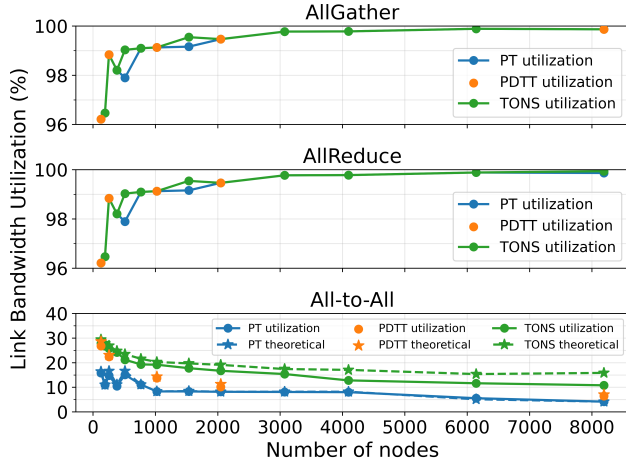


Figure 6: Link utilization of generated schedules for all-gather (top), all-reduce (middle), and all-to-all (bottom) for the best PT (blue), PDDT (orange), and TONS (green). all-gather and all-reduce have a 100% theoretical upper bound; for all-to-all, the limit is derived from MCF (dashed).

of $2.07\times$ over the current baseline. For sizes/configurations without PDDT, TONS has a $2.39\times$ higher saturation point over the best PT and for sizes with PDDT, a $1.65\times$ improvement. These gains not only persist but increase at the largest evaluated sizes (up to 8192 nodes), validating the ability to scale.

Applying AT routing to a baseline torus yields only modest improvements (typically on the order of $\sim 1.1\text{--}1.2\times$). The improvements of AT with respect to DOR are due to VC imbalances as explored in Section 7.4 while the loss (PDDT 2048 and 8192 nodes) is likely due to using the random variant of AT. While AT improves performance marginally, the benefit is fundamentally *topological*. Finally, the TONS LP SYM results match or exceed the non-symmetric LP solutions across large node counts, supporting the claim that symmetry can be used as a scalability lever without sacrificing throughput.

7.2 Collective Communication

We evaluate all-gather, all-reduce, and all-to-all using the scheduling methodology in Section 6.1.2. For simplicity, we evaluate TONS LP SYM and the best PT per size as representative samples. Figure 6 shows that all designs achieve near-ideal utilization for all-gather and all-reduce, consistent with prior observations that these collectives admit highly efficient schedules on regular low-diameter fabrics [38, 45]. Importantly, TONS does not sacrifice performance on these primitives³

³We suspect the few data points of higher utilization are due to diameter and average hop differences affecting the greedy MultiTree [38] algorithm.

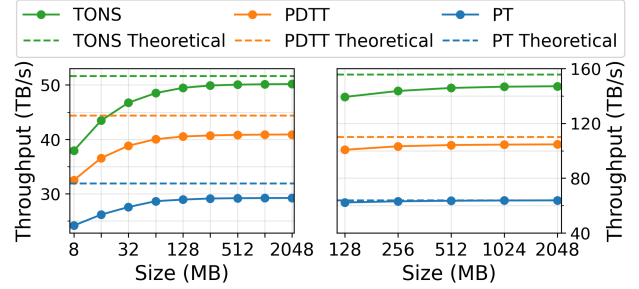


Figure 7: Cumulative network throughput under trace-driven simulation (CNSim) for 256-node (left) and 1024-node (right) PT (blue), PDDT (orange), and TONS (green), sweeping collective/buffer sizes (log scale) until saturation. MCF-based upper bound (dashed) included.

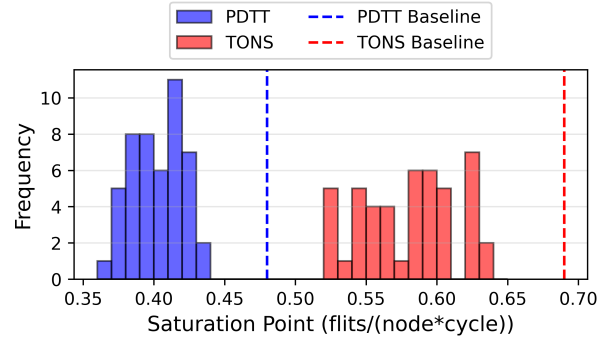


Figure 8: Saturation points for all possible OCS faults for 256 node baseline PDDT WFR (blue) and TONS robust AT (red). The baseline, no OCS fault saturation point for each is shown by the dotted line.

For all-to-all, TONS consistently achieves higher utilization than PT/PDDT, tracking the higher topological (MCF-based) limit, indicating improved concurrent throughput under the most communication-intensive pattern targeted by our synthesis objective. Figure 7 substantiates the schedule-based analysis with trace-driven simulation for two representative topology sizes, 256 and 1024 nodes, where the smallest possible⁴ collectives are not immediately in saturation. The networks soon reach saturation where TONS networks show 9 TB/s and 47 TB/s higher cumulative throughput for 256 and 1024 nodes, respectively, exactly matching analytical expectations.

7.3 Fault Tolerance

The performance of the routing technique for jobs with OCS faults was measured by their saturation points for each of the 48 possible single OCS faults. We compare a 256 node PDDT using WFR (Section 2.2) against TONS using *robust* routing and include each design’s no-fault saturation point as a reference. The histograms of the saturation points are plotted in Figure 8. Figure 8 shows two clear outcomes. First, TONS

⁴128 B flits imply minimums.

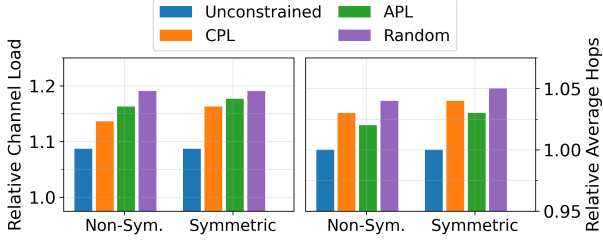


Figure 9: Isolation of AT turns prioritization measuring (left) maximum channel load and (right) average hops (both, lower is better) relative to topology bounds.

sustains substantially higher absolute throughput than PDDT under every fault scenario, indicating that the *robust* AT routing is not sacrificing significant performance. Second, while both designs experience degradation under faults, TONS’s distribution remains centered at a much higher saturation point; qualitatively, TONS shifts the entire fault-throughput, leaving significant headroom even in the degraded regime.

7.4 AT Routing

We isolate the effects of TONS routing choices. We evaluate (i) turn-prioritization heuristics within the AT framework, (ii) the greedy VC load-balancing mechanism, and (iii) the resulting VC utilization relative to DOR on torus baselines.

Turn prioritization and symmetry.

Figure 9 reports maximum channel load (left) and average hops (right), both normalized to topological bounds, for AT variants and a (likely deadlocked) unconstrained routing function. Two takeaways matter for the paper’s claims. First, symmetry-reduced routing preserves throughput. The symmetric and non-symmetric variants achieve nearly identical normalized performance across all prioritization schemes, demonstrating that the large computational savings from symmetry do not impose a measurable throughput penalty in this setting. Second, enforcing deadlock freedom via AT incurs low loss relative to unconstrained routing and the choice of prioritization materially affects that loss. Our preferred turn priority scheme, CPL, only incurs 5% higher concentrated channel load relative to an unconstrained variant. In particular, the CPL heuristic provides a favorable tradeoff, retaining most of the unconstrained throughput while limiting hop inflation. By contrast, weaker prioritization (e.g., random) both reduces throughput and increases hops.

VC load balancing. Static single-path routing can create severe imbalance across virtual channels, which is problematic in low-VC designs where a small number of congested VCs can dominate head-of-line blocking. Figure 10 evaluates hops-per-VC under an intentionally unbalanced assignment versus TONS’s greedy online VC load balancing. The load-balanced variants achieve near-uniform hops-per-VC, indicating that the algorithm effectively spreads long paths and high-volume flows across the available VC budget.

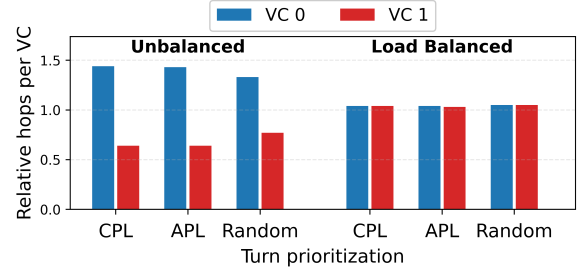


Figure 10: Analytical evaluation of the number of hops per VC between unbalanced and load balanced turn prioritization for 1024 node TONS LP SYM.

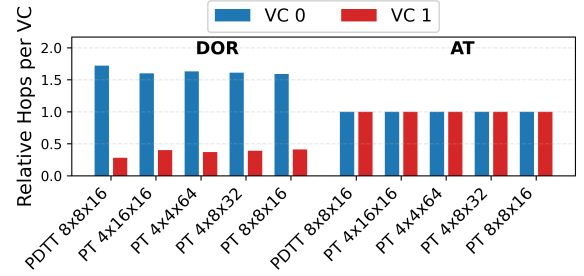


Figure 11: Analytical evaluation of the number of hops per VC for load balanced AT versus DOR.

DOR vs. AT on tori. Finally, we compare VC utilization between traditional DOR and AT on torus baselines. While DOR and AT can have similar max channel load on regular tori, their VC occupancy differs substantially because DOR’s datelineing structure often concentrates traffic into a subset of VCs. Figure 11 shows that DOR skews hops heavily toward VC 0, leaving VC 1 underutilized, whereas AT achieves substantially more balanced VC usage by construction. This balance is especially valuable in the TPU setting, where the VC budget is small and congestion sensitivity is high.

8 CONCLUSION

We demonstrate that optimization-driven network synthesis can produce implementable pod-scale topologies that outperform established structured designs under throughput-oriented demand. TONS generates TPU/OCS-feasible direct networks via LP/ILP/MILP formulations guided by a flow proxy and couples them with deadlock-free static routing under a two-VC budget. Across analytical metrics and cycle-level simulation, TONS delivers higher sustained throughput without sacrificing any form of performance. The resulting designs remain robust under the targeted OCS fault model and our routing/VC assignment avoids pathological load/VC imbalance. Broadly, these results suggest that modern accelerator fabrics have substantial headroom beyond torus families. TONS topologies can be implemented today without hardware changes and adapt modularly to new constraints in next generation computing.

A PROOF OF OPTIMAL MCF USING “ONE-LEG” TRIANGLE INEQUALITY

Let $G = (V, E)$ be a connected undirected graph. Consider the two linear programs:

(LR) *Full metric formulation.* Variables d_{ij} for $i, j \in V$:

$$\begin{aligned} \min_d \quad & \sum_{(i,j) \in E} d_{ij} \\ \text{s.t.} \quad & \sum_{i \in V} \sum_{j \in V} d_{ij} \geq 1, \\ & d_{ij} \leq d_{ik} + d_{kj} \quad \forall i, j, k \in V \text{ distinct}, \\ & d_{ij} = d_{ji} \geq 0 \quad \forall i, j \in V, \\ & d_{ii} = 0 \quad \forall i \in V. \end{aligned}$$

Let d^* be an optimal solution and denote $z_{d^*} := \sum_{(i,j) \in E} d_{ij}^*$.

(LR_OL) *One-leg formulation.* Variables b_{ij} for $i, j \in V$:

$$\begin{aligned} \min_b \quad & \sum_{(i,j) \in E} b_{ij} \\ \text{s.t.} \quad & \sum_{i \in V} \sum_{j \in V} b_{ij} \geq 1, \\ & b_{ij} \leq b_{ik} + b_{kj} \quad \forall i, j, k \in V \text{ distinct with } (i, k) \in E, \\ & b_{ij} = b_{ji} \geq 0 \quad \forall i, j \in V, \\ & b_{ii} = 0 \quad \forall i \in V. \end{aligned}$$

Let b^* be an optimal solution and denote $z_{b^*} := \sum_{(i,j) \in E} b_{ij}^*$. Then

$$z_{b^*} = z_{d^*}.$$

Moreover, from any optimal solution b^* of (LR_OL) one can construct a feasible solution d' of (LR) with $d'_{ij} \geq b_{ij}^*$ for all i, j and $d'_{ij} = b_{ij}^*$ for all $(i, j) \in E$, so d' is optimal for (LR).

We split the argument into two inequalities:

$$z_{b^*} \leq z_{d^*} \quad \text{and} \quad z_{d^*} \leq z_{b^*}.$$

1. Feasible-set inclusion: $z_{b^*} \leq z_{d^*}$.

By definition, d^* satisfies

$$d_{ij}^* \leq d_{ik}^* + d_{kj}^* \quad \forall i, j, k \in V \text{ distinct}.$$

In particular, this holds for all triples with $(i, k) \in E$. Thus d^* satisfies all the triangle inequalities required by (LR_OL), and it clearly satisfies the normalization and symmetry/nonnegativity conditions as well. Hence

d^* is feasible for (LR_OL).

Since (LR_OL) minimizes the same objective over a (weakly) larger feasible region, we obtain

$$z_{b^*} = \min_{b \text{ feasible for (LR_OL)}} \sum_{(i,j) \in E} b_{ij} \leq \sum_{(i,j) \in E} d_{ij}^* = z_{d^*}.$$

2. Shortest-path closure: $z_{d^*} \leq z_{b^*}$.

Let b^* be an optimal solution of (LR_OL). Define edge weights on G by

$$w_{uv} := b_{uv}^* \quad \text{for each } (u, v) \in E.$$

For any $i, j \in V$, define d'_{ij} to be the shortest-path distance from i to j in G with edge weights w :

$$d'_{ij} := \min_{P: i \rightsquigarrow j} \sum_{(u,v) \in P} w_{uv},$$

where the minimum is over all paths P from i to j in G . Since G is connected, every pair has at least one such path, so d'_{ij} is finite.

2.1. Metric property and basic conditions.

By construction, d' is a shortest-path metric on V with nonnegative symmetric weights. In particular,

$$d'_{ij} = d'_{ji}, \quad d'_{ii} = 0, \quad d'_{ij} \geq 0 \quad \forall i, j \in V,$$

and for all $i, j, k \in V$,

$$d'_{ij} \leq d'_{ik} + d'_{kj}.$$

Thus d' satisfies the full triangle inequalities required by (LR).

2.2. One-leg inequalities imply $b_{ij}^* \leq d'_{ij}$.

We first show that for any feasible b of (LR_OL), and for any $i, j \in V$ and any simple path

$$P : i = v_0, v_1, \dots, v_m = j \quad \text{with} \quad (v_r, v_{r+1}) \in E \text{ for all } r,$$

we have

$$b_{ij} \leq \sum_{r=0}^{m-1} b_{v_r v_{r+1}}. \quad (3)$$

We prove (3) by induction on the path length m .

Base case $m = 1$. Then P consists of a single edge (i, j) , and (3) reads

$$b_{ij} \leq b_{ij},$$

which is trivially true.

Inductive step. Assume (3) holds for all paths of length $m - 1$. Now consider a path of length $m \geq 2$:

$$i = v_0, v_1, \dots, v_m = j.$$

Let $k := v_1$. Since $(i, k) = (v_0, v_1) \in E$, the one-leg triangle inequality for (LR_OL) gives

$$b_{ij} \leq b_{ik} + b_{kj}.$$

The suffix $k = v_1, v_2, \dots, v_m = j$ is a path of length $m - 1$, so by the induction hypothesis,

$$b_{kj} \leq \sum_{r=1}^{m-1} b_{v_r v_{r+1}}.$$

Combining these inequalities yields

$$b_{ij} \leq b_{ik} + b_{kj} \leq b_{v_0 v_1} + \sum_{r=1}^{m-1} b_{v_r v_{r+1}} = \sum_{r=0}^{m-1} b_{v_r v_{r+1}},$$

completing the induction.

Applying this to $b = b^*$, and then taking the minimum over all paths from i to j , we obtain

$$b_{ij}^* \leq \min_{P: i \rightsquigarrow j} \sum_{(u,v) \in P} b_{uv}^* = d'_{ij} \quad \forall i, j \in V.$$

Thus

$$b_{ij}^* \leq d'_{ij} \quad \forall i, j \in V. \quad (4)$$

2.3. Normalization for d' .

Since b^* is feasible for (LR_OL), we have

$$\sum_{i \in V} \sum_{j \in V} b_{ij}^* \geq 1.$$

Using (4), we get entrywise $d'_{ij} \geq b_{ij}^*$, hence

$$\sum_{i \in V} \sum_{j \in V} d'_{ij} \geq \sum_{i \in V} \sum_{j \in V} b_{ij}^* \geq 1.$$

Together with the metric property and nonnegativity/symmetry established in 2.1, this shows that d' is feasible for (LR).

2.4. Objective equality on edges.

We now compare d' and b^* on edges. Fix any edge $(i, j) \in E$.

By definition of d' , there exists a path P from i to j realizing the minimum:

$$d'_{ij} = \sum_{(u,v) \in P} b_{uv}^*.$$

In particular, the single-edge path $P = \{(i, j)\}$ is one candidate, so

$$d'_{ij} \leq b_{ij}^*.$$

On the other hand, applying (4) with (i, j) we have

$$b_{ij}^* \leq d'_{ij}.$$

Hence, for all $(i, j) \in E$,

$$d'_{ij} = b_{ij}^*.$$

The objective of both (LR) and (LR_OL) is

$$f(x) = \sum_{(i,j) \in E} x_{ij}.$$

Therefore,

$$f(d') = \sum_{(i,j) \in E} d'_{ij} = \sum_{(i,j) \in E} b_{ij}^* = f(b^*) = z_{b^*}.$$

Since d' is feasible for (LR), optimality of d^* implies

$$z_{d^*} = \min_{d \text{ feasible for (LR)}} f(d) \leq f(d') = z_{b^*}.$$

3. Combining the inequalities.

From Part 1 we have $z_{b^*} \leq z_{d^*}$, and from Part 2 we have $z_{d^*} \leq z_{b^*}$. Thus

$$z_{b^*} = z_{d^*},$$

and d' constructed from b^* is an optimal solution of (LR).

B INTEGRALITY RELAXATION ITERATIVE ALGORITHM

Algorithm 3 Relaxed, Iterative LP

```

1: Input:  $D = (x, y, z, c)$            ▶ Given system dimensions
2: Input:  $interval$                    ▶ Recalculation interval
3: Output:  $M$                          ▶ Complete (binary) topology
4: Initialize  $M \leftarrow \text{electrical\_connections}(D)$ 
5: Initialize  $V \leftarrow \text{valid\_connections}(M)$ 
6: while  $|M| < 6xyz$  do
7:    $\hat{M} \leftarrow \text{LP\_TONS}(M, V, D)$            ▶ Continuous map  $\hat{M}$ 
8:   for  $i < interval$  do
9:      $e \leftarrow \text{max\_optical\_connection}(\hat{M}, V)$ 
10:     $M \leftarrow M \cup e$ 
11:   end for
12: end while

```

C COMPLETE ANALYTICAL METRICS

Size	Topology	Diameter	Avg. Hops	MCF
128	PT 4x4x8	8	4.032	0.00781
	PDTT 4x4x8	6	3.465	0.01364
	TONS MILP	6	3.373	0.01401
	TONS LP	6	3.359	0.01406
	TONS LP SYM	6	3.368	0.01403
192	PT 4x4x12	10	5.026	0.00347
	TONS MILP	6	3.623	0.00867
	TONS LP	6	3.560	0.00882
	TONS LP SYM	6	3.560	0.00883
256	PT 4x8x8	10	5.020	0.00391
	PT 4x4x16	12	6.024	0.00195
	PDTT 4x8x8	6	4.329	0.00544
	TONS MILP	7	3.833	0.00606
	TONS LP	6	3.750	0.00627
	TONS LP SYM	6	3.739	0.00636
384	PT 4x8x12	12	6.016	0.00174
	TONS MILP	7	4.127	0.00380
	TONS LP	7	4.029	0.00389
	TONS LP SYM	6	4.055	0.00392
512	PT 8x8x8	12	6.012	0.00195
	PT 4x8x16	14	7.014	0.00098
	PT 4x4x32	20	10.020	0.00049
	TONS MILP	8	4.384	0.00260
	TONS LP	7	4.258	0.00276
	TONS LP SYM	7	4.259	0.00276
768	PT 8x8x12	14	7.009	0.00087
	TONS LP SYM	7	4.642	0.00169
1024	PT 8x8x16	16	8.008	0.00049
	PT 4x16x16	18	9.009	0.00049
	PT 4x8x32	22	11.011	0.00024
	PT 4x4x64	36	18.018	0.00012
	PDTT 8x8x16	12	6.976	0.00084
	TONS LP SYM	8	4.852	0.00119
1536	PT 8x12x16	18	9.006	0.00033
	PT 8x8x24	20	10.007	0.00022
	PT 4x4x96	52	26.017	0.00005
	TONS LP SYM	8	5.116	0.00077
2048	PT 8x16x16	20	10.005	0.00024
	PDTT 8x16x16	12	8.723	0.00033
	TONS LP SYM	8	5.355	0.00056
3072	PT 12x16x16	22	11.004	0.00016
	PT 4x4x192	100	50.016	0.00001
	TONS LP SYM	9	5.641	0.00034
4096	PT 16x16x16	24	12.003	0.00012
	TONS LP SYM	9	5.877	0.00025
6144	PT 16x16x24	28	14.002	0.00005
	TONS LP SYM	10	6.201	0.00015
8192	PT 16x16x32	32	16.002	0.00003
	PDTT 16x16x32	24	13.986	0.00005
	TONS LP SYM	10	6.464	0.00012

D FEASIBILITY OF OCS FAULT-TOLERANCE

We study when a TPU-style pod network admits $t \geq 2$ *OCS-disjoint spanning trees*. Such a collection implies connectivity under up to $f \leq t - 1$ OCS-domain (color) faults, because at least one tree remains intact when at most f colors fail. We proceed by relating a throughput proxy—the maximum concurrent flow (MCF) value—to the Nash–Williams criterion for the existence of t edge-disjoint spanning trees [60]. Our key step is a conservative lower bound: if the network’s MCF is large enough, then every partition required by Nash–Williams has enough *distinct OCS colors* crossing it, and t OCS-disjoint trees exist.

D.1 Setup and Definitions

Let $G = (V, E)$ be a valid TPU pod graph with $n := |V|$ routers arranged into $\frac{n}{n_c}$ electrical cubes of size $n_c = 64$. We form the *cube graph* $G' = (V', E')$ by contracting each cube into a supernode, so $|V'| = n/n_c$. Edges E' correspond to inter-cube optical connections (OCS links); electrical links remain inside cubes and are not represented in G' .

Cuts and MCF. For any subset $S \subseteq V$, let $\delta_G(S)$ be the set of edges with exactly one endpoint in S and let $|\delta_G(S)|$ be its cardinality. Define the (uniform-demand) *edge expansion* (a.k.a. sparsity)

$$\Phi_G(S) := \frac{|\delta_G(S)|}{|S|(n - |S|)}. \quad (5)$$

Let λ denote the maximum concurrent flow value of G under uniform all-pairs demands and the given edge capacities. A standard cut argument implies that for every nontrivial cut S ,

$$\lambda \leq \Phi_G(S). \quad (6)$$

Equivalently, if we have a certified lower bound $\underline{\lambda}$ on the MCF, then every cut must have at least

$$|\delta_G(S)| \geq \underline{\lambda} |S| (n - |S|). \quad (7)$$

In what follows, we apply (9) to cuts induced by unions of cubes, and we interpret the resulting edge-count lower bounds as lower bounds on the number of OCS links leaving a set of cubes in G' .

OCS colors. Each OCS link in E' is labeled by a *color* (an OCS domain ID) from a set of 48 colors. By construction, across the entire pod each color appears exactly twice (i.e., there are two links of each color), so

$$|E'| = 96 \quad \text{and} \quad \forall \text{ colors } c, |\{e \in E' : \text{col}(e) = c\}| = 2. \quad (8)$$

Consequently, any set of m OCS links must contain at least $\lceil m/2 \rceil$ distinct colors (since one color can contribute at most two links).

D.2 From Nash–Williams to a Throughput and Color Budget Condition

We work with partitions that respect cube boundaries: each part is a union of whole cubes. Let $\mathcal{P} = \{P_0, \dots, P_{k-1}\}$ be such a partition of V into $k \geq 2$ parts and write $p_i := |P_i|$ (routers). Let $C(P_0, \dots, P_{k-1})$ denote the number of *directed* edges of G that cross between distinct parts (equivalently, the sum of directed cut sizes, divided by two to correct double counting).

Nash–Williams. For an undirected graph, the Nash–Williams / Tutte theorem states that G' contains t edge-disjoint spanning trees iff every partition of V' into k parts has at least $t(k - 1)$ inter-part edges [60]. In our setting we strengthen the requirement to *OCS-disjoint* spanning trees, meaning the trees are edge-disjoint and use disjoint sets of OCS colors.

Color-disjoint sufficient condition. A conservative sufficient condition for the existence of t OCS-disjoint spanning trees is: for every cube-respecting partition $\mathcal{P}$ into k parts, there are at least $t(k - 1)$ *distinct OCS colors* on inter-part edges. We lower bound this quantity using the cut guarantee induced by a throughput certificate.

From throughput to distinct inter-part colors. Assume a throughput certificate $\underline{\lambda}$ such that for every part P_i , the directed cut size satisfies

$$|\delta_G(P_i)| \geq \underline{\lambda} p_i (n - p_i). \quad (9)$$

Summing over parts and correcting double counting yields

$$C(P_0, \dots, P_{k-1}) \geq \frac{1}{2} \sum_{i=0}^{k-1} |\delta_G(P_i)| \geq \frac{\underline{\lambda}}{2} \sum_{i=0}^{k-1} p_i (n - p_i). \quad (10)$$

To relate crossing *edges* to crossing *colors*, we use the per-cube wiring constraint (8). In the most adversarial arrangement, each distinct color contributes two directed arcs at a cube, so exposing m directed arcs across a cube-level cut reveals at least $\lceil m/2 \rceil$ distinct colors, but never more than 48. Aggregating this argument across a partition, we obtain the following throughput-driven sufficient condition.

LEMMA D.1 (THROUGHPUT-DRIVEN SUFFICIENT CONDITION). *If*

$$\underline{\lambda} \geq \frac{t}{32n}, \quad (11)$$

then every cube-respecting partition $\mathcal{P}$ satisfies the strengthened Nash–Williams requirement of at least $t(k - 1)$ distinct inter-part OCS colors. Consequently, G' admits t OCS-disjoint spanning trees.

Color budget saturation and the effective upper bound on t . The condition (11) captures a *throughput* limitation. Independently, OCS-disjoint trees cannot share colors and there

are only 48 colors, so

$$t \leq 48. \quad (12)$$

Combining (11) and (12), the maximum number of OCS-disjoint spanning trees that can be certified from $\underline{\lambda}$ is

$$t_{\max} \leq \min \{ \lfloor 32n\underline{\lambda} \rfloor, 48 \}. \quad (13)$$

This explains why a topology may satisfy $32n\underline{\lambda} > 48$ (suggesting more than 48 trees from the throughput bound alone) while still being limited to at most 48 OCS-disjoint trees by the finite color alphabet.

Connectivity under f color faults. Let a color fault remove all edges of a failed color from G' . Because the t spanning trees are OCS-disjoint, any single color fault can destroy edges from at most one tree; therefore, under f color faults at most f trees are destroyed and at least $t - f$ trees remain intact. If $f \leq t - 1$, then at least one spanning tree survives, which implies cube-to-cube connectivity. Since routers within a cube remain electrically connected, this implies that every router can still reach every other router in the pod.

Thus, a sufficient condition for tolerating up to f color faults is the existence of $t \geq f + 1$ OCS-disjoint spanning trees. Using (13), a conservative throughput-and-budget condition is

$$\underline{\lambda} \geq \frac{f + 1}{32n} \quad \text{and} \quad f \leq 47. \quad (14)$$

Empirical check. In our experiments, we report both terms in (13): the throughput-implied count $\lfloor 32n\underline{\lambda} \rfloor$ and the hard cap of 48 colors. This makes clear when fault tolerance is limited by global throughput versus by the finite OCS color budget.

REFERENCES

- [1] [n. d.]. TPU v4. <https://cloud.google.com/tpu/docs/v4>. ([n. d.]). [Accessed 11-09-2025].
- [2] 2026. TPU v5p. <https://docs.cloud.google.com/tpu/docs/v5p>. (2026). [Accessed 11-09-2025].
- [3] Hussam Abu-Libdeh, Paolo Costa, Antony Rowstron, Greg O'Shea, and Austin Donnelly. 2010. Symbiotic routing in future data centers. In *Proceedings of the ACM SIGCOMM 2010 conference*. 51–62.
- [4] Paul Barham, Aakanksha Chowdhery, Jeff Dean, Sanjay Ghemawat, Steven Hand, Daniel Hurt, Michael Isard, Hyeontaek Lim, Ruoming Pang, Sudip Roy, et al. 2022. Pathways: Asynchronous distributed dataflow for ml. *Proceedings of Machine Learning and Systems* 4 (2022), 430–449.
- [5] Prithwish Basu, Liangyu Zhao, Jason Fantl, Siddharth Pal, Arvind Krishnamurthy, and Joud Khoury. 2024. Efficient all-to-all collective communication schedules for direct-connect topologies. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*. 28–41.
- [6] Maciej Besta and Torsten Hoefler. 2014. Slim fly: A cost effective low-diameter network topology. In *SC'14: proceedings of the international conference for high performance computing, networking, storage and analysis*. IEEE, 348–359.
- [7] Christian Blikl, Pierre Bonami, and Andrea Lodi. 2014. Solving mixed-integer quadratic programming problems with IBM-CPLEX: a progress report. In *Proceedings of the twenty-sixth RAMP symposium*. 16–17.
- [8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. (2020). arXiv:cs.CL/2005.14165 <https://arxiv.org/abs/2005.14165>
- [9] Jose M Camara, Miquel Moreto, Enrique Vallejo, Ramon Beivide, Jose Miguel-Alonso, Carmen Martinez, and Javier Navaridas. 2010. Twisted torus topologies for enhanced interconnection networks. *IEEE Transactions on Parallel and Distributed Systems* 21, 12 (2010), 1765–1778.
- [10] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. PaLM: Scaling Language Modeling with Pathways. (2022). arXiv:cs.CL/2204.02311 <https://arxiv.org/abs/2204.02311>
- [11] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. Mscclang: Microsoft collective communication language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 502–514.
- [12] Andrew R. Curtis, Tommy Carpenter, Mustafa Elsheikh, Alejandro López-Ortiz, and S. Keshav. 2012. REWIRE: An optimization-based framework for unstructured data center network design. In *2012 Proceedings IEEE INFOCOM*. 1116–1124. <https://doi.org/10.1109/INFOCOM.2012.6195470>
- [13] Dally and Seitz. 1987. Deadlock-Free Message Routing in Multiprocessor Interconnection Networks. *IEEE Trans. Comput.* C-36, 5 (1987), 547–553. <https://doi.org/10.1109/TC.1987.1676939>
- [14] William James Dally and Brian Patrick Towles. 2004. *Principles and practices of interconnection networks*. Elsevier.
- [15] Jens Domke, Torsten Hoefler, and Satoshi Matsuoka. 2016. Routing on the Dependency Graph: A New Approach to Deadlock-Free High-Performance Routing. In *Proceedings of the 25th ACM International Symposium on High-Performance Parallel and Distributed Computing (HPDC '16)*. Association for Computing Machinery, New York, NY, USA, 3–14. <https://doi.org/10.1145/2907294.2907313>
- [16] Jens Domke, Torsten Hoefler, and Wolfgang E. Nagel. 2011. Deadlock-Free Oblivious Routing for Arbitrary Topologies. In *2011 IEEE International Parallel and Distributed Processing Symposium*. 616–627. <https://doi.org/10.1109/IPDPS.2011.65>

- [17] Yuanyuan Dong, Eli V Olinick, T Jason Kratz, and David W Matula. 2015. A compact linear programming formulation of the maximum concurrent flow problem. *Networks* 65, 1 (2015), 68–87.
- [18] J. Duato. 1995. A necessary and sufficient condition for deadlock-free adaptive routing in wormhole networks. *IEEE Transactions on Parallel and Distributed Systems* 6, 10 (1995), 1055–1067. <https://doi.org/10.1109/71.473515>
- [19] Jack Edmonds. 1965. Maximum matching and a polyhedron with 0, 1-vertices. *Journal of research of the National Bureau of Standards B* 69, 125-130 (1965), 55–56.
- [20] Anup Gangwar et. al. 2020. Automated Synthesis of Custom Networks-on-Chip for Real World Applications. In *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD '20)*. Association for Computing Machinery, New York, NY, USA, Article 41, 9 pages. <https://doi.org/10.1145/3400302.3415656>
- [21] Jason Lowe-Power et. al. 2020. The gem5 Simulator: Version 20.0+. CoRR abs/2007.03152 (2020). arXiv:2007.03152 <https://arxiv.org/abs/2007.03152>
- [22] Natalie Enright Jerger et. al. 2014. NoC Architectures for Silicon Interposer Systems: Why Pay for more Wires when you Can Get them (from your interposer) for Free?. In *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. 458–470. <https://doi.org/10.1109/MICRO.2014.61>
- [23] Sangeetha Abdu Jyothi et. al. 2016. Measuring and Understanding Throughput of Network Topologies. In *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 761–772. <https://doi.org/10.1109/SC.2016.64>
- [24] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiah Fainman, George Papen, and Amin Vahdat. 2010. Helios: a hybrid electrical/optical switch architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*. 339–350.
- [25] Yinxiao Feng, Yuchen Wei, Dong Xiang, and Kaisheng Ma. 2024. Evaluating Chiplet-based Large-Scale Interconnection Networks via Cycle-Accurate Packet-Parallel Simulation. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 731–747.
- [26] Jared Fernandez, Luca Wehrstedt, Leonid Shamis, Mostafa Elhoushi, Kalyan Saladi, Yonatan Bisk, Emma Strubell, and Jacob Kahn. 2025. Efficient Hardware Scaling and Diminishing Returns in Large-Scale Training of Language Models. *Transactions on Machine Learning Research* (2025).
- [27] Christopher J. Glass and Lionel M. Ni. 1992. The Turn Model for Adaptive Routing. In *Proceedings of the 19th Annual International Symposium on Computer Architecture (ISCA '92)*. Association for Computing Machinery, New York, NY, USA, 278–287. <https://doi.org/10.1145/139669.140384>
- [28] Conor James Green and Mithuna Thottethodi. 2024. NetSmith: An Optimization Framework for Machine-Discovered Network Topologies. In *Proceedings of the 53rd International Conference on Parallel Processing (ICPP '24)*. Association for Computing Machinery, New York, NY, USA, 421–432. <https://doi.org/10.1145/3673038.3673060>
- [29] Albert Greenberg, James R. Hamilton, Navendu Jain, Srikanth Kandula, Changhoon Kim, Parantap Lahiri, David A. Maltz, Parveen Patel, and Sudipta Sengupta. 2009. VL2: A Scalable and Flexible Data Center Network. In *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication (SIGCOMM '09)*. Association for Computing Machinery, New York, NY, USA, 51–62. <https://doi.org/10.1145/1592568.1592576>
- [30] Chen Griner, Johannes Zerwas, Andreas Blenk, Manya Ghobadi, Stefan Schmid, and Chen Avin. 2021. Cerberus: The power of choices in datacenter topology design—a throughput perspective. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 5, 3 (2021), 1–33.
- [31] Chuanxiong Guo, Guohan Lu, Dan Li, Haitao Wu, Xuan Zhang, Yunfeng Shi, Chen Tian, Yongguang Zhang, and Songwu Lu. 2009. BCube: a high performance, server-centric network architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2009 conference on Data communication*. 63–74.
- [32] Chuanxiong Guo, Haitao Wu, Kun Tan, Lei Shi, Yongguang Zhang, and Songwu Lu. 2008. Dcell: a scalable and fault-tolerant network structure for data centers. In *Proceedings of the ACM SIGCOMM 2008 conference on Data communication*. 75–86.
- [33] Gurobi Optimization, LLC. 2023. Gurobi Optimizer Reference Manual. (2023). <https://www.gurobi.com>
- [34] László Gyarmati and Tuan Anh Trinh. 2010. Scafida: A scale-free network inspired data center architecture. *ACM SIGCOMM Computer Communication Review* 40, 5 (2010), 4–12.
- [35] Juris Hartmanis. 1982. Computers and Intractability: A Guide to the Theory of NP-Completeness (Michael R. Garey and David S. Johnson). *SIAM Rev.* 24, 1 (1982), 90–91. <https://doi.org/10.1137/1024022>
- [36] Joel Hestness and Stephen W Keckler. 2011. Netrace: Dependency-tracking traces for efficient network-on-chip experimentation. *The University of Texas at Austin, Dept. of Computer Science, Tech. Rep* (2011).
- [37] Yuanfang Hu, Yi Zhu, Hongyu Chen, Ronald Graham, and Chung-Kuan Cheng. 2006. Communication latency aware low power NoC synthesis. In *Proceedings of the 43rd annual Design Automation Conference*. 574–579.
- [38] Jiayi Huang, Pritam Majumder, Sungkeun Kim, Abdullah Muzahid, Ki Hwan Yum, and Eun Jung Kim. 2021. Communication Algorithm-Architecture Co-Design for Distributed Deep Learning. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 181–194. <https://doi.org/10.1109/ISCA52012.2021.00023>
- [39] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyoukjoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).
- [40] Imase and Itoh. 1983. A design for directed graphs with minimum diameter. *IEEE Trans. Comput.* 100, 8 (1983), 782–784.
- [41] Chenyu Jiang, Ye Tian, Zhen Jia, Shuai Zheng, Chuan Wu, and Yida Wang. 2024. Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping. *Proceedings of Machine Learning and Systems* 6 (2024), 74–86.
- [42] Nan Jiang, Daniel U. Becker, George Michelogiannakis, James Balfour, Brian Towles, D. E. Shaw, John Kim, and William J. Dally. 2013. A detailed and flexible cycle-accurate Network-on-Chip simulator. In *2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 86–96. <https://doi.org/10.1109/ISPASS.2013.6557149>
- [43] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. 2020. A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 463–479. <https://www.usenix.org/conference/osdi20/presentation/jjiang>
- [44] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. 2024. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 745–760.
- [45] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian

- Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–14.
- [46] Michael Jünger, Gerhard Reinelt, and Stefan Thienel. 1995. Practical problem solving with cutting plane algorithms in combinatorial optimization. *DIMACS series in discrete mathematics and theoretical computer science* 20, 1995 (1995), 111–152.
- [47] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [48] W Kautz. 1968. Bounds on directed (d, k) graphs. *Theory of cellular logic networks and machines, Final Report* (1968), 20–28.
- [49] Tom Leighton and Satish Rao. 1999. Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. *J. ACM* 46, 6 (Nov. 1999), 787–832. <https://doi.org/10.1145/331524.331526>
- [50] Hong Liu, Ryohei Urata, Kevin Yasumura, Xiang Zhou, Roy Bannon, Jill Berger, Pedram Dashti, Norm Jouppi, Cedric Lam, Sheng Li, et al. 2023. Lightwave fabrics: at-scale optical circuit switching for datacenter and machine learning systems. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 499–515.
- [51] Hong Liu, Ryohei Urata, Kevin Yasumura, Xiang Zhou, Roy Bannon, Jill Berger, Pedram Dashti, Norm Jouppi, Cedric Lam, Sheng Li, et al. 2024. Reconfigurable Lightwave Fabrics for ML Supercomputers. In *2024 Optical Fiber Communications Conference and Exhibition (OFC)*. IEEE, 1–3.
- [52] O. Lysne, T. Skeie, S.-A. Reinemo, and I. Theiss. 2006. Layered routing in irregular networks. *IEEE Transactions on Parallel and Distributed Systems* 17, 1 (2006), 51–65. <https://doi.org/10.1109/TPDS.2006.12>
- [53] Ben Mann, Nick Ryder, Melanie Subbiah, J Kaplan, P Dhariwal, A Neelakantan, P Shyam, G Sastry, A Askell, S Agarwal, et al. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* 1, 3 (2020), 3.
- [54] David W. Matula and Farhad Shahrokhi. 1990. Sparsest cuts and bottlenecks in graphs. *Discrete Applied Mathematics* 27, 1 (1990), 113–123. [https://doi.org/10.1016/0166-218X\(90\)90133-W](https://doi.org/10.1016/0166-218X(90)90133-W)
- [55] William M Mellette, Rob McGuinness, Arjun Roy, Alex Forencich, George Papen, Alex C Snoeren, and George Porter. 2017. Rotornet: A scalable, low-complexity, optical datacenter network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 267–280.
- [56] AI Meta. 2025. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, checked on 4, 7 (2025), 2025.
- [57] Jayaram Mudigonda, Praveen Yalagandula, and Jeffrey C Mogul. 2011. Taming the flying cable monster: A topology design and optimization framework for {Data-Center} networks. In *2011 USENIX Annual Technical Conference (USENIX ATC 11)*.
- [58] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.
- [59] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–15.
- [60] C St JA Nash-Williams. 1961. Edge-disjoint spanning trees of finite graphs. *Journal of the London Mathematical Society* 1, 1 (1961), 445–450.
- [61] Viet Hung Nguyen and Michel Minoux. 2021. Linear size MIP formulation of Max-Cut: new properties, links with cycle inequalities and computational results. *Optimization Letters* 15, 4 (2021), 1041–1060.
- [62] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Lukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Lukasz Kondruciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rameev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin

- Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. (2024). arXiv:cs.CL/2303.08774 <https://arxiv.org/abs/2303.08774>
- [63] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, Rishi Kapoor, Stephen Kratzer, Nanfang Li, Hong Liu, Karthik Nagaraj, Jason Ornstein, Samir Sawhney, Ryohei Urata, Lorenzo Vicisano, Kevin Yasumura, Shidong Zhang, Junlan Zhou, and Amin Vahdat. 2022. Jupiter evolving: transforming google’s datacenter network via optical circuit switches and software-defined networking. In *Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM ’22)*. Association for Computing Machinery, New York, NY, USA, 66–85. <https://doi.org/10.1145/3544216.3544265>
- [64] Ragheb Rahmaniani, Teodor Gabriel Crainic, Michel Gendreau, and Walter Rei. 2017. The Benders decomposition algorithm: A literature review. *European Journal of Operational Research* 259, 3 (2017), 801–817.
- [65] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. (2020). arXiv:cs.LG/1910.02054 <https://arxiv.org/abs/1910.02054>
- [66] Yves Robert, Sameer Shende, Allen Malony, Alan Morris, Wyatt Spear, Scott Biersdorff, Burton Smith, Dali Wang, Daniel Ricciuto, Wilfred Post, Michael Berry, François Irigoin, Katherine Yelick, S.L. Graham, Paul Hilfinger, Dan Bonachea, Amir Kamil, Kaushik Datta, and J. Moss. 2011. *Encyclopedia of Parallel Computing*. 2025–2029. https://doi.org/10.1007/978-0-387-09766-4_59
- [67] Brandon Schlinker, Radhika Niranjana Mysore, Sean Smith, Jeffrey C Mogul, Amin Vahdat, Minlan Yu, Ethan Katz-Bassett, and Michael Rubin. 2015. Condor: Better topologies through declarative design. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 449–463.
- [68] Alexander Schrijver. 1998. *Theory of linear and integer programming*. John Wiley & Sons.
- [69] Alexander Sergeev and Mike Del Balso. 2018. Horovod: fast and easy distributed deep learning in TensorFlow. (2018). arXiv:cs.LG/1802.05799 <https://arxiv.org/abs/1802.05799>
- [70] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 593–612.
- [71] Farhad Shahrokhi and D. W. Matula. 1990. The maximum concurrent flow problem. *J. ACM* 37, 2 (April 1990), 318–334. <https://doi.org/10.1145/77600.77620>
- [72] Ji-Yong Shin, Bernard Wong, and Emin Gün Sirer. 2011. Small-world datacenters. In *Proceedings of the 2nd ACM Symposium on Cloud Computing (SOCC ’11)*. Association for Computing Machinery, New York, NY, USA, Article 2, 13 pages. <https://doi.org/10.1145/2038916.2038918>
- [73] Mohammad Shoeibi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. (2020). arXiv:cs.CL/1909.08053 <https://arxiv.org/abs/1909.08053>
- [74] Arnav Shukla, Harsh Sharma, Srikant Bharadwaj, Vinayak Abrol, and Sujay Deb. 2025. Taming the Tail: NoI Topology Synthesis for Mixed DL Workloads on Chiplet-Based Accelerators. (2025). arXiv:cs.AR/2510.24113 <https://arxiv.org/abs/2510.24113>
- [75] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannon, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, et al. 2015. Jupiter rising: A decade of clos topologies and centralized control in google’s datacenter network. *ACM SIGCOMM computer communication review* 45, 4 (2015), 183–197.
- [76] Ankit Singla, P. Brighten Godfrey, and Alexandra Kolla. 2014. High Throughput Data Center Topology Design. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. USENIX Association, Seattle, WA, 29–41. <https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/singla>
- [77] Ankit Singla, Chi-Yao Hong, Lucian Popa, and P. Brighten Godfrey. 2012. Jellyfish: Networking Data Centers Randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. USENIX Association, San Jose, CA, 225–238. <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/singla>
- [78] K. Srinivasan, K.S. Chatha, and G. Konjevod. 2006. Linear-programming-based techniques for synthesis of network-on-chip architectures. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 14, 4 (2006), 407–420. <https://doi.org/10.1109/TVLSI.2006.871762>
- [79] Lawrence C Stewart and David Gingold. 2006. A new generation of cluster interconnect. *White Paper*, SiCortex Inc (2006).
- [80] Min Yee Teh, Shizhen Zhao, Peirui Cao, and Keren Bergman. 2020. COUDER: Robust Topology Engineering for Optical Circuit Switched Data Center Networks. (2020). arXiv:cs.NI/2010.00090 <https://arxiv.org/abs/2010.00090>
- [81] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. 2022. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239* (2022).
- [82] Brian Towles, William J Dally, and Stephen Boyd. 2003. Throughput-centric routing algorithm design. In *Proceedings of the fifteenth annual ACM symposium on Parallel algorithms and architectures*. 200–209.
- [83] Ryohei Urata, Hong Liu, Kevin Yasumura, Erji Mao, Jill Berger, Xiang Zhou, Cedric Lam, Roy Bannon, Darren Hutchinson, Daniel Nelson, et al. 2022. Mission Apollo: Landing optical circuit switching at data-center scale. *arXiv preprint arXiv:2208.10041* (2022).
- [84] Amin Vahdat and Mark Lohmeyer. 2023. Enabling next-generation AI workloads: Announcing TPU v5p and AI Hypercomputer. <https://cloud.google.com/blog/products/ai-machine-learning/introducing-cloud-tpu-v5p-and-ai-hypercomputer>. (2023). [Accessed 11-09-2025].
- [85] Asaf Valadarsky, Gal Shahaf, Michael Dinitz, and Michael Schapira. 2016. Xpander: Towards optimal-performance datacenters. In *Proceedings of the 12th International Conference on emerging Networking EXperiments and Technologies*. 205–219.
- [86] Leslie G. Valiant. 1982. A scheme for fast parallel communication. *SIAM journal on computing* 11, 2 (1982), 350–361.
- [87] Guohui Wang, David G Andersen, Michael Kaminsky, Konstantina Papagiannaki, TS Eugene Ng, Michael Kozuch, and Michael Ryan. 2010. c-Through: Part-time optics in data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*. 327–338.
- [88] Yongji Wu, Yechen Xu, Jingrong Chen, Zhaodong Wang, Ying Zhang, Matthew Lentz, and Danyang Zhuo. 2024. MCCS: A Service-based Approach to Collective Communication for Multi-Tenant Cloud. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 679–690.
- [89] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, Ruoming Pang, Noam Shazeer, Shibo Wang, Tao Wang, Yonghui Wu, and Zhifeng Chen. 2021. GSPMD: General and Scalable Parallelization for ML Computation Graphs. (2021). arXiv:cs.DC/2105.04663 <https://arxiv.org/abs/2105.04663>

- [90] Ye Yu and Chen Qian. 2016. Space Shuffle: A Scalable, Flexible, and High-Performance Data Center Network. *IEEE Transactions on Parallel and Distributed Systems* 27, 11 (2016), 3351–3365. <https://doi.org/10.1109/TPDS.2016.2533618>
- [91] Rui Zhang-Shen and Nick McKeown. 2008. Designing a fault-tolerant network using valiant load-balancing. In *IEEE INFOCOM 2008-The 27th Conference on Computer Communications*. IEEE, 2360–2368.
- [92] Liangyu Zhao, Siddharth Pal, Tapan Chugh, Weiyang Wang, Jason Fantl, Prithwish Basu, Joud Khoury, and Arvind Krishnamurthy. 2025. Efficient {Direct-Connect} Topologies for Collective Communications. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 705–737.
- [93] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.
- [94] Yazhou Zu, Alireza Ghaffarkhah, Hoang-Vu Dang, Brian Towles, Steven Hand, Safeen Huda, Adekunle Bello, Alexander Kolbasov, Arash Rezaei, Dayou Du, Steve Lacy, Hang Wang, Aaron Wisner, Chris Lewis, and Henri Bahini. 2024. Resiliency at Scale: Managing Google’s TPUv4 Machine Learning Supercomputer. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 761–774. <https://www.usenix.org/conference/nsdi24/presentation/zu>